SENG 319: Software Design Patterns

Group Project 1: Detection and Refactoring of Anti-Patterns and Code Smells
Group Name: TOZARING | Target Codebase: CheckStyle 14.1.0

Part I: Software Design Anti-Patterns

1. Anti-Pattern Types (Design Smells) Detected by PTIDEJ

Using the PTIDEJ analysis tool on the CheckStyle codebase, all 16 design smell detectors located in lesson-vm-out/design smells were evaluated. The table below lists all 16 design smell types and their detected instance counts:

Anti-Pattern / Design Smell Type	Instances	Source File (lesson-vm-out/design smells/)
RefusedParentBequest	40	RefusedParentBequest.ini
ComplexClass	31	ComplexClass.ini
LongParameterList	19	LongParameterList.ini
LazyClass	13	LazyClass.ini
Blob	7	Blob.ini
ClassDataShouldBePrivate	7	ClassDataShouldBePrivate.ini
AntiSingleton	3	AntiSingleton.ini
BaseClassShouldBeAbstract	2	BaseClassShouldBeAbstract.ini
SpeculativeGenerality	2	SpeculativeGenerality.ini
BaseClassKnowsDerivedClass	0	BaseClassKnowsDerivedClass.txt
FunctionalDecomposition	0	FunctionalDecomposition.txt
LargeClass	0	LargeClass.txt
ManyFieldAttributesButNotComplex	0	ManyFieldAttributesButNotComplex.ini
MessageChains	0	MessageChains.ini
SwissArmyKnife	0	SwissArmyKnife.ini
TraditionBreaker	0	TraditionBreaker.ini
Total Design Smells (Anti-Patterns)	**124**	**Across 16 files in lesson-vm-out/design smells/**

2. Most Impactful Anti-Pattern on CheckStyle Quality & Justification

From the PTIDEJ detection results in (1), multiple design smells significantly degrade software quality in CheckStyle. Non-private class data (ClassDataShouldBePrivate) compromises encapsulation, and excessively long argument lists (LongParameterList) harm developer ergonomics and code clarity. However, Blob (7 instances) and AntiSingleton (3 instances) have the most profound negative impact on overall architectural integrity, maintainability, memory usage, and runtime reliability:

- [Blob (God Class) - Architectural, Memory & Maintainability Impact]: Blob classes (such as SarifLogger, Checker, JavadocMethodCheck, and AuditEventDefaultFormatter paired with DataClass AuditEvent) monopolize processing logic over passive DataClass collaborators. In Blob #4 (AuditEventDefaultFormatter + AuditEvent), AuditEvent acts purely as a passive data holder exposing raw getters while AuditEventDefaultFormatter exhibits Feature Envy via disconnected static helpers (getCheckShortName and initStringBuilderWithOptimalBuffer). Furthermore, format() invoked getCheckShortName(event) twice per violation event, duplicating substring heap allocations and

miscalculating StringBuilder capacity whenever moduleId was configured.

- [AntiSingleton - Concurrency, State Safety & Runtime Impact]: The AntiSingleton anti-pattern occurs when a class exposes non-final static fields that function as unprotected global mutable state. In JavaLanguageParser (generated from JavaLanguageParser.g4), the package-private static field "static int fileCounter = 0;" is mutated inside the parser constructor every time CheckStyle parses a Java file to decide when to clear the ANTLR DFA state cache (_interp.clearDFA()). Because this mutable static primitive is neither private nor synchronized, multi-threaded file checking triggers race conditions (lost increments or concurrent resets), leading to non-deterministic DFA cache clearing and memory/performance degradation.

3. Refactoring Two Anti-Pattern Instances

Instance 1: AntiSingleton in com.puppycrawl.tools.checkstyle.grammar.java.JavaLanguageParser

a. Reason for Choosing the Anti-Pattern Instance:

We chose the AntiSingleton instance in JavaLanguageParser because it sits directly on CheckStyle's hot path (instantiated once per parsed Java source file) and resolves both an object-oriented encapsulation violation and a real thread-safety race condition with a clean, self-contained in-place refactoring. Note that the other two AntiSingleton instances reported in AntiSingleton.ini (JavaLanguageLexer and JavadocCommentsLexer) stem from ANTLR 4's built-in Java code generator template (Java.stg), which emits "public static String[] channelNames" and "public static String[] modeNames" without the "final" modifier into target/generated-sources/antlr. In contrast, JavaLanguageParser.fileCounter is hand-written code inside JavaLanguageParser.g4.

b. Location Within the CheckStyle Codebase [Original Code Snippet]:

File: src/main/resources/com/puppycrawl/tools/checkstyle/grammar/java/JavaLanguageParser.g4 (lines 54-78):

```
    static int fileCounter = 0;

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        fileCounter++;
        if (fileCounter > clearDfaLimit) {
            _interp.clearDFA();
            fileCounter = 0;
        }
    }
```

c. Steps to Refactor and Solve the Issue & Justification:

- Step 1 (Encapsulate Mutable Counter in DfaClearCounter): Inside @parser::members of JavaLanguageParser.g4, we defined the static final helper class DfaClearCounter that encapsulates "private final int limit" and the mutable instance field "private int count".
- Step 2 (Replace Mutable Static Field with Private Static Final Constant): We removed "static int fileCounter = 0;" and replaced it in place with "private static final DfaClearCounter DFA_CLEAR_COUNTER = new DfaClearCounter(CLEAR_DFA_LIMIT);". Because the static reference is private and final, JavaLanguageParser no longer exposes a mutable static global variable.
- Step 3 (Synchronize State Transitions): We implemented synchronized tick() and tick(int customLimit) methods on DfaClearCounter so that incrementing count, comparing against the DFA limit, and resetting count to 0 happen atomically in a single critical section, eliminating data races during concurrent file parsing.

d. Refactored Code Snippet & PTIDEJ Verification Result:

```
    private static final DfaClearCounter DFA_CLEAR_COUNTER =
            new DfaClearCounter(CLEAR_DFA_LIMIT); // Anti-Pattern Refactor

    public static final class DfaClearCounter { // Anti-Pattern Refactor
        private final int limit; // Anti-Pattern Refactor
        private int count; // Anti-Pattern Refactor

        public DfaClearCounter(int limit) { // Anti-Pattern Refactor
            this.limit = limit; // Anti-Pattern Refactor
        }

        public synchronized boolean tick() { // Anti-Pattern Refactor
            return tick(limit); // Anti-Pattern Refactor
        }

        public synchronized boolean tick(int customLimit) { // Anti-Pattern Refactor
            final boolean shouldClear; // Anti-Pattern Refactor
            if (++count > customLimit) { // Anti-Pattern Refactor
                count = 0; // Anti-Pattern Refactor
                shouldClear = true; // Anti-Pattern Refactor
            }
            else {
                shouldClear = false; // Anti-Pattern Refactor
            }
            return shouldClear; // Anti-Pattern Refactor
        }
    }

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        if (DFA_CLEAR_COUNTER.tick(clearDfaLimit)) { // Anti-Pattern Refactor
            _interp.clearDFA(); // Anti-Pattern Refactor
        }
    }
```

PTIDEJ Resolution Result: Because "static int fileCounter" has been removed and replaced with a "private static final" reference whose mutable state is an encapsulated private instance variable inside DfaClearCounter, the AntiSingleton smell is completely resolved in JavaLanguageParser.

Instance 2: Blob (#4) in AuditEventDefaultFormatter & AuditEvent

a. Reason for Choosing the Anti-Pattern Instance:

We chose Blob #4 (com.puppycrawl.tools.checkstyle.AuditEventDefaultFormatter paired with com.puppycrawl.tools.checkstyle.api.AuditEvent) for two compelling reasons: (1) AuditEvent is the most recurrent collaborator across all detected Blobs in Blob.ini (appearing in Blob #1, Blob #2, and Blob #4), and (2) inspecting AuditEventDefaultFormatter revealed that its uncohesive static helper getCheckShortName(AuditEvent) was not only exhibiting Feature Envy on AuditEvent, but was being called TWICE for every formatted violation once inside initStringBuilderWithOptimalBuffer() and a second time inside format() allocating redundant substring objects on the heap and miscalculating buffer capacity whenever moduleId was present.

b. Location Within the CheckStyle Codebase [Original Code Snippets]:

File: src/main/java/com/puppycrawl/tools/checkstyle/AuditEventDefaultFormatter.java (lines 65-117):

```
        final StringBuilder sb = initStringBuilderWithOptimalBuffer(event, severityLevelName);
        ...
        if (event.getModuleId() == null) {
            final String checkShortName = getCheckShortName(event);
            sb.append(checkShortName);
        }
        else {
            sb.append(event.getModuleId());
        }
        ...
    private static StringBuilder initStringBuilderWithOptimalBuffer(AuditEvent event,
            String severityLevelName) {
        final int bufLen = LENGTH_OF_ALL_SEPARATORS + event.getFileName().length()
            + event.getMessage().length() + severityLevelName.length()
            + getCheckShortName(event).length();
        return new StringBuilder(bufLen);
    }

    private static String getCheckShortName(AuditEvent event) {
        final String checkFullName = event.getSourceName();
        String checkShortName = checkFullName.substring(checkFullName.lastIndexOf('.') + 1);
        if (checkShortName.endsWith(SUFFIX)) {
            final int endIndex = checkShortName.length() - SUFFIX.length();
            checkShortName = checkShortName.substring(0, endIndex);
        }
        return checkShortName;
    }
```

c. Steps to Refactor and Solve the Issue & Justification:

- Step 1 (Move Method to AuditEvent): We moved the domain logic for deriving the stripped short check name (getCheckShortName(String suffix)) and resolving the effective module display identifier (getSourceIdentifier(String suffix)) directly into AuditEvent.java. Because this logic depends exclusively on AuditEvent's own sourceName and moduleId, placing it on AuditEvent cures Feature Envy and enriches AuditEvent beyond a passive data container.
- Step 2 (Remove Uncohesive Static Helper & Eliminate Duplicate Allocations): We removed getCheckShortName(AuditEvent) from AuditEventDefaultFormatter and updated format(AuditEvent) to call event.getSourceIdentifier(checkSuffix) exactly ONCE. This eliminates duplicate String.substring() heap allocations on every log event and sizes the StringBuilder capacity using the true sourceIdentifier length.
- Step 3 (Restore Class Cohesion): We introduced cohesive instance attributes (separatorLength and checkSuffix) initialized in AuditEventDefaultFormatter's constructor and accessed directly by format(AuditEvent). Sharing these instance fields across the constructor and formatting logic, combined with removing disconnected static helpers, fully restores class cohesion.

d. Refactored Code Snippets & PTIDEJ Verification Result:

1. In src/main/java/com/puppycrawl/tools/checkstyle/api/AuditEvent.java:

```
    public String getCheckShortName(String suffix) { // Anti-Pattern Refactor
        final String checkFullName = getSourceName();
        final int lastDotIndex = checkFullName.lastIndexOf('.');
        String checkShortName = checkFullName.substring(lastDotIndex + 1);
        if (checkShortName.endsWith(suffix)) {
            final int endIndex = checkShortName.length() - suffix.length();
            checkShortName = checkShortName.substring(0, endIndex);
        }
        return checkShortName;
    }

    public String getSourceIdentifier(String suffix) { // Anti-Pattern Refactor
        final String moduleId = getModuleId();
        final String identifier;
        if (moduleId == null) {
            identifier = getCheckShortName(suffix);
        }
        else {
            identifier = moduleId;
        }
        return identifier;
    }
```

2. In src/main/java/com/puppycrawl/tools/checkstyle/AuditEventDefaultFormatter.java:

```
    private final int separatorLength; // Anti-Pattern Refactor
    private final String checkSuffix; // Anti-Pattern Refactor

    public AuditEventDefaultFormatter() {
        separatorLength = LENGTH_OF_ALL_SEPARATORS; // Anti-Pattern Refactor
        checkSuffix = SUFFIX; // Anti-Pattern Refactor
    }

    @Override
    public String format(AuditEvent event) {
        ...
        final String sourceIdentifier = event.getSourceIdentifier(checkSuffix); // Anti-Pattern
Refactor
        final int bufLen = separatorLength + fileName.length()
                + message.length() + severityLevelName.length()
                + sourceIdentifier.length(); // Anti-Pattern Refactor
        final StringBuilder sb = new StringBuilder(bufLen); // Anti-Pattern Refactor
        ...
        sb.append(": ").append(message)
            .append(" [").append(sourceIdentifier).append(']'); // Anti-Pattern Refactor
        return sb.toString();
    }
```

PTIDEJ Resolution Result: By removing the uncohesive static helper getCheckShortName from AuditEventDefaultFormatter, sharing instance attributes across constructor and format(), and moving domain behavior into AuditEvent, AuditEventDefaultFormatter achieves high structural cohesion and AuditEvent operates as a rich domain entity, completely resolving Blob #4.

Part II: Code Smells

1. Code Smells Detected by PTIDEJ

Using the PTIDEJ analysis tool on the CheckStyle codebase, all 33 code smell detectors located in lesson-vm-out/code smells were evaluated. The table below lists all 33 code smell types and their detected instance counts:

Code Smell Type	Instances	Source File (lesson-vm-out/code smells/)

ParentClassProvidesProtectedDetection	434	ParentClassProvidesProtectedDetection.ini
MethodNoParameterDetection	415	MethodNoParameterDetection.ini
NotAbstract	415	NotAbstract.ini
NoPolymorphism	401	NoPolymorphism.ini
TwoInheritanceDetection	327	TwoInheritanceDetection.ini
NotComplexClassDetection	197	NotComplexClassDetection.ini
RareOverridingDetection	163	RareOverridingDetection.ini
LongMethodClassDetection	135	LongMethodClassDetection.ini
ChildClassDetection	131	ChildClassDetection.ini
FieldPrivate	128	FieldPrivate.ini
NoInheritanceDetection	107	NoInheritanceDetection.ini
NotComplexDetection	94	NotComplexDetection.ini
ComplexClassOnlyDetection	49	ComplexClassOnlyDetection.ini
LowCohesionDetection	42	LowCohesionDetection.ini
LowCohesionOnlyDetection	42	LowCohesionOnlyDetection.ini
DataClass	36	DataClass.ini
LargeClassDetection	31	LargeClassDetection.ini
LargeClassOnlyDetection	31	LargeClassOnlyDetection.ini
ManyAttributesDetection	30	ManyAttributesDetection.ini
ClassOneMethod	25	ClassOneMethod.ini
HasChildrenDetection	22	HasChildrenDetection.ini
AbstractClass	19	AbstractClass.ini
LongParameterListClassDetection	19	LongParameterListClassDetection.ini
LongMethodDetection	16	LongMethodDetection.ini
FewMethodsDetection	13	FewMethodsDetection.ini
ControllerClassDetection	7	ControllerClassDetection.ini
FieldPublic	7	FieldPublic.ini
OneChildClassDetection	5	OneChildClassDetection.ini
ClassGlobalVariable	3	ClassGlobalVariable.ini
FunctionClassDetection	3	FunctionClassDetection.ini
NotClassGlobalVariable	3	NotClassGlobalVariable.ini
MessageChainsClassDetection	0	MessageChainsClassDetection.ini
MultipleInterface	0	MultipleInterface.ini
Total Code Smells	**3350**	**Across 33 files in lesson-vm-out/code smells/**

2. Selected Code Smells and Justification

From the 3,350 detected Code Smell instances across 33 categories, several smells negatively influence code quality and software evolution. FieldPublic (7 instances) directly breaks data encapsulation and exposes

internal state. ClassOneMethod (25 instances) bloats class structures with single-purpose utility wrappers that should be simplified or merged. However, we selected OneChildClassDetection (5 instances) and LongMethodClassDetection (135 instances) as the most critical and impactful smells to refactor:

- [OneChildClass (Unnecessary Hierarchies) - Architectural Bloat & Indirection]: OneChildClass occurs when an abstract base class exists with exactly one concrete subclass. These single-child abstract classes provide no polymorphic value, adding artificial inheritance layers, extra source files, and conceptual overhead without any second implementation ever existing. Refactoring via Collapse Hierarchy cleanly eliminates unnecessary indirection, simplifying codebase navigation and maintenance.
- [LongMethodClass (Bloated Methods) - Maintainability & Cognitive Complexity]: LongMethodClass methods pack excessive procedural logic, multiple branch paths, and overlapping local variables into a single block. These methods become severe maintenance liabilities because understanding, reviewing, testing, or updating any individual concern requires comprehending the entire monolith. Decomposing them via Extract Method into focused, well-named helper methods restores readability and testability.

3. Refactoring of the Selected Code Smells

Smell A: OneChildClass in xpath Package (AbstractRootNode and AbstractElementNode)

a. Reason for Choosing the Code Smell Instances:

We targeted OneChildClass.2 (AbstractRootNode -> RootNode) and OneChildClass.5 (AbstractElementNode -> ElementNode) from OneChildClassDetection.ini. Both were abstract classes in com.puppycrawl.tools.checkstyle.xpath with only one child implementation each. They added needless file count and hierarchy depth without serving any polymorphism.

b. Location Within the CheckStyle Codebase [Code Snippets]:

Files: src/main/java/com/puppycrawl/tools/checkstyle/xpath/AbstractRootNode.java (178 lines) and AbstractElementNode.java (324 lines), extended only by RootNode.java and ElementNode.java.

c. Steps to Refactor and Solve the Issue & Justification:

- Step 1 (Collapse Hierarchy - Root Node): All members of AbstractRootNode (constants, compareOrder, getAttributeValue, getLocalPart, getNodeKind, getParent, getRoot, iterateAxis, getDepth) were moved verbatim into RootNode. RootNode now extends AbstractNode directly, and AbstractRootNode.java was safely deleted.
- Step 2 (Collapse Hierarchy - Element Node): All members of AbstractElementNode were merged into ElementNode, and the single abstract hook createAttributeNode() became a private helper method since no other class overrides it. ElementNode now extends AbstractNode directly, and AbstractElementNode.java was safely deleted.

d. Refactored Code Snippets & PTIDEJ Verification Result:

```
// In src/main/java/com/puppycrawl/tools/checkstyle/xpath/RootNode.java:
public class RootNode extends AbstractNode { // Code Smell Refactor: extends AbstractNode direct
  ly
    public RootNode(DetailAST detailAst) {
        super(new GenericTreeInfo(Configuration.newConfiguration()));
        this.detailAst = detailAst;
    }
    ... // all methods and constants consolidated from AbstractRootNode
}

// In src/main/java/com/puppycrawl/tools/checkstyle/xpath/ElementNode.java:
public class ElementNode extends AbstractNode { // Code Smell Refactor: extends AbstractNode dir
  ectly
    public ElementNode(AbstractNode root, AbstractNode parent, DetailAST detailAst,
            int depth, int indexAmongSiblings) {
        super(root.getTreeInfo());
        ...
    }
    private AttributeNode createAttributeNode() { ... } // former abstract hook converted to pri
  vate
    ... // all methods and constants consolidated from AbstractElementNode
}
```

PTIDEJ Resolution Result: Because AbstractRootNode and AbstractElementNode no longer exist and their functionality is folded directly into their concrete implementations, both OneChildClass instances in the xpath package are eliminated from OneChildClassDetection.ini.

Smell B: LongMethodClass in ImportControlLoader (startElement)

a. Reason for Choosing the Code Smell Instance:

We targeted LongMethodClass.93 in com.puppycrawl.tools.checkstyle.checks.imports.ImportControlLoader (from LongMethodClassDetection.ini). The method startElement() was a monolithic SAX callback handling multiple distinct XML element types (import-control, subpackage, file, and import rules) within a single method body, mixing XML attribute parsing with control-stack manipulation.

b. Location Within the CheckStyle Codebase [Code Snippets]:

File: src/main/java/com/puppycrawl/tools/checkstyle/checks/imports/ImportControlLoader.java (lines 174-208):

```
    public void startElement(String namespaceUri, String localName,
                             String qualifiedName, Attributes attributes) throws SAXException {
        if ("import-control".equals(qualifiedName)) {
            final String pkg = safeGet(attributes, PKG_ATTRIBUTE_NAME);
            final MismatchStrategy strategyOnMismatch = getStrategyForImportControl(attributes);
            final boolean regex = containsRegexAttribute(attributes);
            stack.push(new PkgImportControl(pkg, regex, strategyOnMismatch));
        }
        else if (SUBPACKAGE_ELEMENT_NAME.equals(qualifiedName)) { ... }
        else if (FILE_ELEMENT_NAME.equals(qualifiedName)) { ... }
        else { ... }
    }
```

c. Steps to Refactor and Solve the Issue & Justification:

- Step 1 (Extract Element-Handling Methods): Using the Extract Method refactoring, each branch of startElement() was extracted into a focused, self-contained private method: startImportControlElement(attributes), startSubpackageElement(attributes), and startFileElement(attributes).
- Step 2 (Simplify startElement into Clean Dispatcher): The startElement() method was reduced to a concise dispatcher that routes SAX events to the appropriate handler, drastically reducing its cognitive complexity and making each tag-parsing routine independently testable and readable.

d. Refactored Code Snippet & PTIDEJ Verification Result:

```
    @Override
    public void startElement(String namespaceUri, String localName,
                             String qualifiedName, Attributes attributes) throws SAXException {
        // Code Smell Refactor: clean dispatcher replacing monolithic method
        if ("import-control".equals(qualifiedName)) {
            startImportControlElement(attributes);
        }
        else if (SUBPACKAGE_ELEMENT_NAME.equals(qualifiedName)) {
            startSubpackageElement(attributes);
        }
        else if (FILE_ELEMENT_NAME.equals(qualifiedName)) {
            startFileElement(attributes);
        }
        else {
            final AbstractImportRule rule = createImportRule(qualifiedName, attributes);
            stack.peek().addImportRule(rule);
        }
    }

    private void startImportControlElement(Attributes attributes) throws SAXException { ... }
    private void startSubpackageElement(Attributes attributes) throws SAXException { ... }
    private void startFileElement(Attributes attributes) throws SAXException { ... }
```

PTIDEJ Resolution Result: The method startElement() in ImportControlLoader is now a concise 15-line dispatcher, removing the excessive length and complexity flagged under LongMethodClassDetection.ini.